

13 Introduction to Array

11/8/25

day 28

Date :
Page No. :

Why? : without array find max marks.

int $S_1 = 67$, $S_2 = 70$ --- $S_{100} = 50$;

int max = S_1 ;

if ($S_2 > \text{max}$)

max = S_2

if (---)

;

100 time if loops.

∴ Data Structure comes & Array's also come.

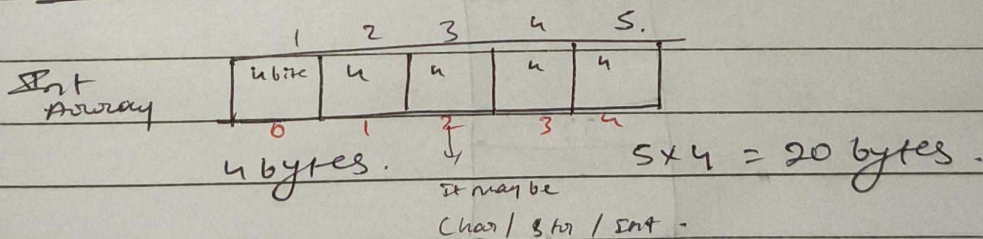
In C, C++, Java.

↳ In this languages the Arrays will be

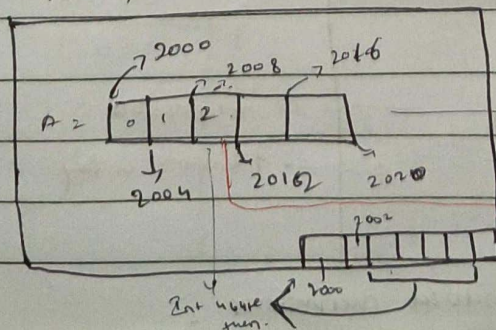
↳ Homogeneous set of elements.

↳ means same data type elements only.

Arrays



RAM



SN:- $A[2] \rightarrow 2 \times 4 = 8.$

base address = 2000

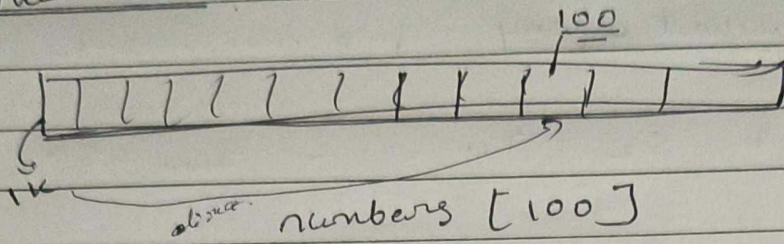
2000

+ 8

2008

this encurs.

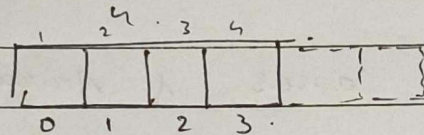
numbers



numbers [100] = 1 # change the number

Array disadvantage

L> Fixed size.



L> If you add 5th element it not possible.

L> & also it not sorted.

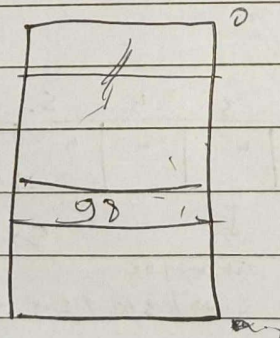
Arrays in different language

In C++ → Vector

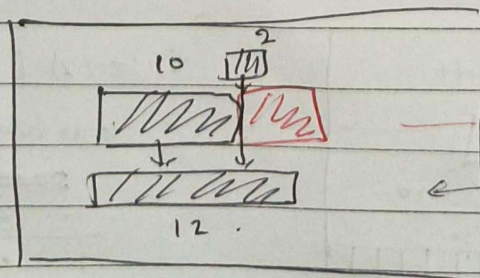
python → list

Java → Array list

this are dynamic array.



RAM



→ It not possible.

← that's why

drawback :- Extra memory

2D Array

		0	1	2	3	4	→ Columns
numbers	→ 0	0,0	0,1	0,2	0,3	0,4	
rows	→ 1	1,0	1,1				
	2						

for read

numbers [0, 0]

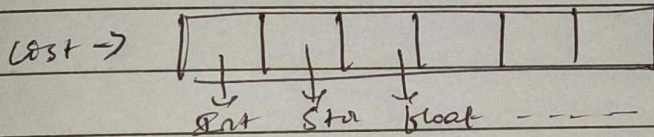
for write.

numbers [1, 3] = 10.

In python Lists

↳ This is having the means.

In the list we can add different Data types.



The data type are different but how it manage.

↳ There are using pointers

pointers :- remembering the address of the variable. value

pointers

1000	0	In the address 101 → value
12000	1	
1000	2	
-1	3	
{	4	
{	5	

sm with Abstract Classes.py

differencebw.py

nestedlistimp.py

test

06_Data Structure in Python > 01_List in Python workshop > List.py > ...

```
1 numbers = [1, 2, "raju", 4, 5, 2.54]
```

```
2
```

```
3 print(numbers[0])
```

```
4 print(numbers[1])
```

```
5 print(numbers[2])
```

```
6
```

```
7 print(numbers)
```

```
8
```

```
9 numbers[0] = 10
```

```
10 print(numbers)
```

```
11
```

```
12 numbers = [1, 2, 3]
```

```
1 numbers = [1, 4, 9, 11, 5]
2
3 numbers.append(6)
4 numbers.append(7)
5 numbers.append(12)
6
7 print(numbers)
8
9 numbers.sort()
10 print(numbers)
11
12 numbers.insert(2, 8)
13 print(numbers)
14
15 numbers.remove(4)
16 print(numbers)
17
18 numbers.reverse()
19 print(numbers)
20
21 value = numbers.pop()
22 print(value)
23 print(numbers)
24
25 for number in numbers:
26     print(number)
27
28 print(numbers[3])
29
30
```

```
30
31     # 2D
32
33     matrix = [
34         [1, 2, 3],
35         [4, 5, 6]
36     ]
37
38     print(matrix)
39
40     matrix[0][1] = 10
41     print(matrix)
42
```